Class: TE **Subject: AISC** **Sem-V**

Experiment No: 07

Course Outcome: Implement supervised learning algorithms for classification.
Blooms Level: L3 – Apply

Aim: To implement a supervised learning algorithm using Multi-Layer Perceptron (MLP) and predict the output for given input data.

Abstract: MLP is a feed-forward artificial neural network used for supervised learning. It consists of an input layer, one or more hidden layers, and an output layer. During training, the network learns the relationship between input features and labeled outputs by adjusting its weights. In this experiment, a simple XOR dataset is used to train an MLP classifier and generate predictions.

Sample Input and Output:
Case 1: Input [0, 0] → Output 0
Case 2: Input [0, 1] → Output 1
Case 3: Input [1, 0] → Output 1
Case 4: Input [1, 1] → Output 0

Theory:
Multi-Layer Perceptron (MLP) is a supervised learning model inspired by the structure of a biological neural network. An MLP contains an input layer, hidden layer(s), and an output layer. Each neuron receives inputs, applies weights and a bias, and passes the result through an activation function. During training, the model minimizes prediction error by updating its weights.
For classification, the MLPClassifier from Scikit-learn can be used. The model is trained using the fit() function and predictions are obtained using predict (). In this experiment, the XOR problem is used because it cannot be solved by a single-layer perceptron, while an MLP with a hidden layer can learn the required non-linear relationship.

Program:

```
from sklearn.neural_network import MLPClassifier

# Input data
X = [[0, 0],
     [0, 1],
     [1, 0],
     [1, 1]]

# Target output (XOR)
y = [0, 1, 1, 0]

# Create MLP model
mlp = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='tanh',
    solver='lbfgs',
    max_iter=1000,
    random_state=1
)

# Train the model
mlp.fit(X, y)

# Predict output
prediction = mlp.predict(X)

# Display results
print("Input:")
print(X)
```

```
print("Actual Output:")
print(y)

print("Predicted Output:")
print(prediction)
```

Output: Input:
[[0, 0], [0, 1], [1, 0], [1, 1]]
Actual Output:
[0, 1, 1, 0]
Predicted Output:
[0 1 1 0]

Code:

https://colab.research.google.com/drive/1kHebsYVn_VAYTHh5ZLcQaVscQeowmr5t?usp=sharing

Conclusion:
The Multi-Layer Perceptron (MLP) supervised learning algorithm was successfully implemented. The model learned the non-linear XOR relationship from the labeled training data and correctly predicted the outputs.

Exercise 1:

A college wants to predict whether a student will Pass or Fail based on the number of hours studied.

Consider the following training data:

Hours Studied	Result
1	Fail
2	Fail
3	Fail
5	Pass
6	Pass
7	Pass

Implement a supervised learning algorithm in Python to learn from this data and predict the result for a new student.

Tasks
Create the given training dataset.
Train a MLP classifier.
Accept the number of hours studied by a new student.
Predict and display:
Pass, or Fail
Display the training accuracy.

HINT:
X = input feature → hours studied
y = target/output → Pass (1) or Fail (0)
model.fit() = trains the supervised learning model

Class: TE **Subject: AISC** **Sem-V**

model.predict() = predicts the result for a new student
model.score() = calculates training accuracy

Exercise 2: Implement a Multi-Layer Perceptron (MLP) classifier using Python for the Iris dataset. Train the model using suitable training and testing data, predict the flower class for test samples, and evaluate the model using accuracy and confusion matrix.

Abstract: This exercise focuses on applying an MLP classifier to the Iris dataset. The model is trained and tested using sample data, and its performance is evaluated using accuracy and a confusion matrix to demonstrate supervised classification.

Class: TE **Subject: AISC** **Sem-V**

